JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang.invoke

Interface MethodHandleInfo

`public interface MethodHandleInfo`

A symbolic reference obtained by cracking a direct method handle into its consitutent symbolic parts. To crack a direct method handle, call `Lookup.revealDirect`.

Direct Method Handles

A *direct method handle* represents a method, constructor, or field without any intervening argument bindings or other transformations. The method, constructor, or field referred to by a direct method handle is called its *underlying member*. Direct method handles may be obtained in any of these ways:

- By executing an `ldc` instruction on a `CONSTANT_MethodHandle` constant. (See the Java Virtual Machine Specification, sections 4.4.8 and 5.4.3.)
- By calling one of the Lookup Factory Methods, such as `Lookup.findVirtual`, to resolve a symbolic reference into a method handle. A symbolic reference consists of a class, name string, and type.
- By calling the factory method `Lookup.unreflect` or `Lookup.unreflectSpecial` to convert a `Method` into a method handle.
- By calling the factory method `Lookup.unreflectConstructor` to convert a `Constructor` into a method handle.
- By calling the factory method `Lookup.unreflectGetter` or `Lookup.unreflectSetter` to convert a `Field` into a method handle.

Restrictions on Cracking

Given a suitable `Lookup` object, it is possible to crack any direct method handle to recover a symbolic reference for the underlying method, constructor, or field. Cracking must be done via a `Lookup` object equivalent to that which created the

target method handle, or which has enough access permissions to recreate an equivalent method handle.

If the underlying method is caller sensitive, the direct method handle will have been "bound" to a particular caller class, the lookup class of the lookup object used to create it. Cracking this method handle with a different lookup class will fail even if the underlying method is public (like `Class.forName`).

The requirement of lookup object matching provides a "fast fail" behavior for programs which may otherwise trust erroneous revelation of a method handle with symbolic information (or caller binding) from an unexpected scope. Use `MethodHandles.reflectAs(java.lang.Class<T>, java.lang.invoke.MethodHandle)` to override this limitation.

Reference kinds

The Lookup Factory Methods correspond to all major use cases for methods, constructors, and fields. These use cases may be distinguished using small integers as follows:

reference kind	descriptive name	scope	member	behavior
1	REF_getField	class	`FT f;`	`(T) this.f;`
2	REF_getStatic	class or interface	`static FT f;`	`(T) C.f;`
3	REF_putField	class	`FT f;`	`this.f = x;`
4	REF_putStatic	class	`static FT f;`	`C.f = arg;`
5	REF_invokeVirtual	class	`T m(A*);`	`(T) this.m(arg*);`
6	REF_invokeStatic	class or interface	`static T m(A*);`	`(T) C.m(arg*);`
7	REF_invokeSpecial	class or interface	`T m(A*);`	`(T) super.m(arg*);`
8	REF_newInvokeSpecial	class	`C(A*);`	`new C(arg*);`
9	REF_invokeInterface	interface	`T m(A*);`	`(T) this.m(arg*);`

Since:

1.8

Field Summary

Fields

Modifier and Type	Field	Description

static int	REF_getField	A direct method handle reference kind, as defined in the **table above**.
static int	REF_getStatic	A direct method handle reference kind, as defined in the **table above**.
static int	REF_invokeInterface	A direct method handle reference kind, as defined in the **table above**.
static int	REF_invokeSpecial	A direct method handle reference kind, as defined in the **table above**.
static int	REF_invokeStatic	A direct method handle reference kind, as defined in the **table above**.
static int	REF_invokeVirtual	A direct method handle reference kind, as defined in the **table above**.
static int	REF_newInvokeSpecial	A direct method handle reference kind, as defined in the **table above**.
static int	REF_putField	A direct method handle reference kind, as defined in the **table above**.
static int	REF_putStatic	A direct method handle reference kind, as defined in the **table above**.

Method Summary

Abstract Methods

Default Methods

Modifier and Type	Method	Description
Class<?>	getDeclaringClass()	Returns the class in which the cracked method handle's

		underlying member was defined.
MethodType	**getMethodType**()	Returns the nominal type of the cracked symbolic reference, expressed as a method type.
int	**getModifiers**()	Returns the access modifiers of the underlying member.
String	**getName**()	Returns the name of the cracked method handle's underlying member.
int	**getReferenceKind**()	Returns the reference kind of the cracked method handle, which in turn determines whether the method handle's underlying member was a constructor, method, or field.
default boolean	**isVarArgs**()	Determines if the underlying member was a variable arity method or constructor.
static **String**	**referenceKindToString**(int referenceKind)	Returns the descriptive name of the given reference kind, as defined in the **table above**.

<T extends **Member**> T	**reflectAs**(**Class**<T> expected, **MethodHandles.Lookup** lookup)	Reflects the underlying member as a method, constructor, or field object.
static **String**	**toString**(int kind, **Class**<?> defc, **String** name, **MethodType** type)	Returns a string representation for a `MethodHandleInfo`, given the four parts of its symbolic reference.

Field Detail

REF_getField

`static final int REF_getField`

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_getStatic

`static final int REF_getStatic`

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_putField

`static final int REF_putField`

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_putStatic

```
static final int REF_putStatic
```

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_invokeVirtual

```
static final int REF_invokeVirtual
```

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_invokeStatic

```
static final int REF_invokeStatic
```

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_invokeSpecial

```
static final int REF_invokeSpecial
```

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_newInvokeSpecial

```
static final int REF_newInvokeSpecial
```

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

REF_invokeInterface

```
static final int REF_invokeInterface
```

A direct method handle reference kind, as defined in the table above.

See Also:

Constant Field Values

Method Detail

getReferenceKind

```
int getReferenceKind()
```

Returns the reference kind of the cracked method handle, which in turn determines whether the method handle's underlying member was a constructor, method, or field. See the table above for definitions.

Returns:

the integer code for the kind of reference used to access the underlying member

getDeclaringClass

```
Class<?> getDeclaringClass()
```

Returns the class in which the cracked method handle's underlying member was defined.

Returns:

the declaring class of the underlying member

getName

```
String getName()
```

Returns
the name of the cracked method handle's underlying member. This is `"<init>"` if the underlying member was a constructor, else it is a simple method name or field name.

Returns:

the simple name of the underlying member

getMethodType

```
MethodType getMethodType()
```

Returns the nominal type of the cracked symbolic reference, expressed as a method type. If the reference is to a constructor, the return type will be `void`. If it is to a non-static method, the method type will not mention the `this` parameter. If it is to a field and the requested access is to read the field, the method type will have no parameters and return the field type. If it is to a field and the requested access is to write the field, the method type will have one parameter of the field type and return `void`.

Note that original direct method handle may include a leading `this` parameter, or (in the case of a constructor) will replace the `void` return type with the constructed class. The nominal type does not include any `this` parameter, and (in the case of a constructor) will return `void`.

Returns:

the type of the underlying member, expressed as a method type

reflectAs

```
<T extends Member> T reflectAs(Class<T> expected,
                               MethodHandles.Lookup lookup)
```

Reflects the underlying member as a method, constructor, or field object. If the underlying member is public, it is reflected as if by `getMethod`, `getConstructor`, or `getField`. Otherwise, it is reflected as if by `getDeclaredMethod`, `getDeclaredConstructor`, or `getDeclaredField`. The underlying member must be accessible to the given lookup object.

Type Parameters:

T - the desired type of the result, either Member or a subtype

Parameters:

expected - a class object representing the desired result type T

lookup - the lookup object that created this MethodHandleInfo, or one with equivalent access privileges

Returns:

a reference to the method, constructor, or field object

Throws:

ClassCastException - if the member is not of the expected type

NullPointerException - if either argument is null

IllegalArgumentException - if the underlying member is not accessible to the given lookup object

getModifiers

```
int getModifiers()
```

Returns the access modifiers of the underlying member.

Returns:

the Java language modifiers for underlying member, or -1 if the member cannot be accessed

See Also:

Modifier, reflectAs(java.lang.Class<T>, java.lang.invoke.MethodHandles.Lookup)

isVarArgs

```
default boolean isVarArgs()
```

Determines if the underlying member was a variable arity method or constructor. Such members are represented by method handles that are varargs collectors.

Implementation Requirements:

This produces a result equivalent to:

```
getReferenceKind() >= REF_invokeVirtual && Modifier.isTransient(getModifiers
```

Returns:

true if and only if the underlying member was declared with variable arity.

referenceKindToString

```
static String referenceKindToString(int referenceKind)
```

Returns the descriptive name of the given reference kind, as defined in the table above. The conventional prefix "REF_" is omitted.

Parameters:

`referenceKind` - an integer code for a kind of reference used to access a class member

Returns:

a mixed-case string such as "getField"

Throws:

`IllegalArgumentException` - if the argument is not a valid reference kind number

toString

```
static String toString(int kind,
                       Class<?> defc,
                       String name,
                       MethodType type)
```

Returns a string representation for a `MethodHandleInfo`, given the four parts of its symbolic reference. This is defined to be of the form `"RK C.N:MT"`, where `RK` is the reference kind string for `kind`, `C` is the name of `defc` `N` is the `name`, and `MT` is the `type`. These four values may be obtained from the reference kind, declaring class, member name, and method type of a `MethodHandleInfo` object.

Implementation Requirements:

This produces a result equivalent to:

```
String.format("%s %s.%s:%s", referenceKindToString(kind), defc.getName(), na
```

Parameters:

`kind` - the reference kind part of the symbolic reference

`defc` - the declaring class part of the symbolic reference

`name` - the member name part of the symbolic reference

`type` - the method type part of the symbolic reference

Returns:

a string of the form "RK C.N:MT"

Throws:

`IllegalArgumentException` - if the first argument is not a valid reference kind number

`NullPointerException` - if any reference argument is null

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.